



**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO
PAULISTA: ENGENHARIA DE DADOS**

Data Engineering Programming

Trabalho Final Projeto Pyspark

SÃO PAULO
2025

29ABD

Bruno Gomes Nogueira – 364457

Evaldo Loiola Mota Junior – 364056

Herivelto Raimuindo Lemos de Macedo Junior – 364212

João Paulo Martins Rodrigues – 364668

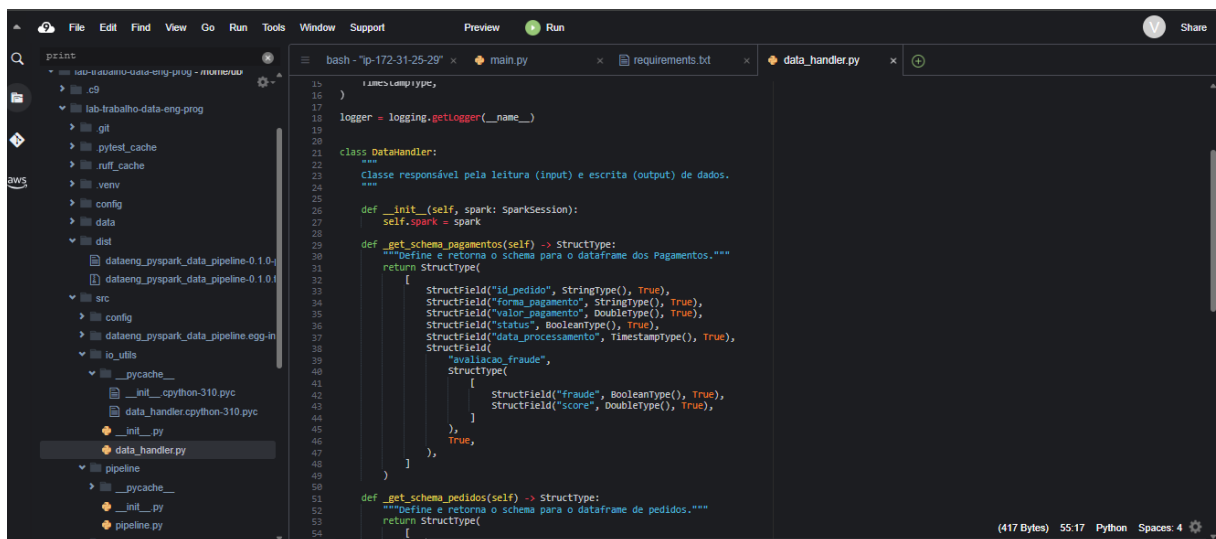


Nesse trabalho, optamos por seguir duas linhas de desenvolvimento. Sendo elas, uma com base no [material de apoio](#) e outra seguindo um padrão de estrutura mais compacto.

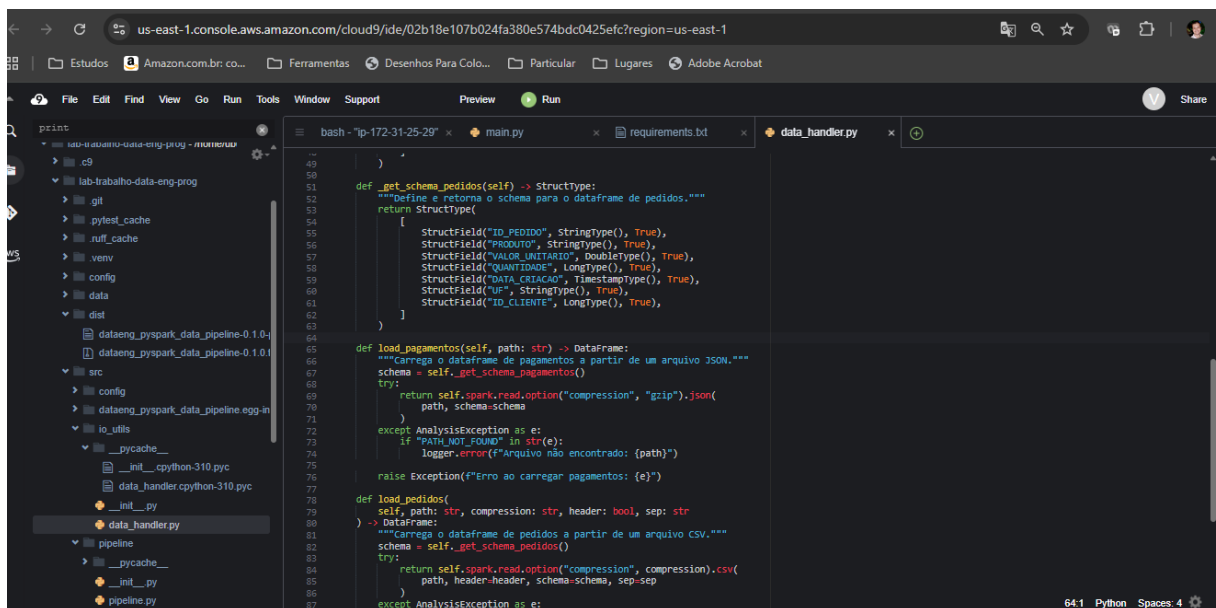
Desenvolvimento conforme material de apoio

Github entrega: <https://github.com/BrunoGomesNogueira/aws-cloud9-trabalho>

1. **Schemas explícitos**



```
15 )
16
17 logger = logging.getLogger(__name__)
18
19
20
21 class DataHandler:
22     """
23     Classe responsável pela leitura (input) e escrita (output) de dados.
24     """
25
26     def __init__(self, spark: SparkSession):
27         self.spark = spark
28
29
30     def _get_schema_pagamentos(self) -> StructType:
31         """Define e retorna o schema para o dataframe dos Pagamentos."""
32         return StructType(
33             [
34                 StructField("id_pedido", StringType(), True),
35                 StructField("forma_pagamento", StringType(), True),
36                 StructField("valor_pagamento", DoubleType(), True),
37                 StructField("status", BooleanType(), True),
38                 StructField("data_processamento", TimestampType(), True),
39                 StructField(
40                     "avaliacao_fraude",
41                     StructType(
42                         [
43                             StructField("fraude", BooleanType(), True),
44                             StructField("score", DoubleType(), True),
45                         ]
46                     ),
47                     True,
48                 ),
49             ]
50         )
51
52     def _get_schema_pedidos(self) -> StructType:
53         """Define e retorna o schema para o dataframe de pedidos."""
54         return StructType(
55             [
56                 StructField("ID_PEDIDO", StringType(), True),
57                 StructField("PRODUTO", StringType(), True),
58                 StructField("VALOR_UNITARIO", DoubleType(), True),
59                 StructField("QUANTIDADE", LongType(), True),
60                 StructField("DATA_CRIACAO", TimestampType(), True),
61                 StructField("UF", StringType(), True),
62                 StructField("ID_CLIENTE", LongType(), True),
63             ]
64         )
65
66     def load_pagamentos(self, path: str) -> DataFrame:
67         """Carrega o dataframe de pagamentos a partir de um arquivo JSON."""
68         schema = self._get_schema_pagamentos()
69         try:
70             return self.spark.read.option("compression", "gzip").json(
71                 path, schema=schema
72             )
73         except AnalysisException as e:
74             if "PATH_NOT_FOUND" in str(e):
75                 logger.error(f"Arquivo não encontrado: {path}")
76             raise Exception(f"Erro ao carregar pagamentos: {e}")
77
78     def load_pedidos(
79         self, path: str, compression: str, header: bool, sep: str
80     ) -> DataFrame:
81         """Carrega o dataframe de pedidos a partir de um arquivo CSV."""
82         schema = self._get_schema_pedidos()
83         try:
84             return self.spark.read.option("compression", compression).csv(
85                 path, header=header, schema=schema, sep=sep
86             )
87         except AnalysisException as e:
```



```
49
50
51
52     def _get_schema_pedidos(self) -> StructType:
53         """Define e retorna o schema para o dataframe de pedidos."""
54         return StructType(
55             [
56                 StructField("ID_PEDIDO", StringType(), True),
57                 StructField("PRODUTO", StringType(), True),
58                 StructField("VALOR_UNITARIO", DoubleType(), True),
59                 StructField("QUANTIDADE", LongType(), True),
60                 StructField("DATA_CRIACAO", TimestampType(), True),
61                 StructField("UF", StringType(), True),
62                 StructField("ID_CLIENTE", LongType(), True),
63             ]
64         )
65
66     def load_pagamentos(self, path: str) -> DataFrame:
67         """Carrega o dataframe de pagamentos a partir de um arquivo JSON."""
68         schema = self._get_schema_pagamentos()
69         try:
70             return self.spark.read.option("compression", "gzip").json(
71                 path, schema=schema
72             )
73         except AnalysisException as e:
74             if "PATH_NOT_FOUND" in str(e):
75                 logger.error(f"Arquivo não encontrado: {path}")
76             raise Exception(f"Erro ao carregar pagamentos: {e}")
77
78     def load_pedidos(
79         self, path: str, compression: str, header: bool, sep: str
80     ) -> DataFrame:
81         """Carrega o dataframe de pedidos a partir de um arquivo CSV."""
82         schema = self._get_schema_pedidos()
83         try:
84             return self.spark.read.option("compression", compression).csv(
85                 path, header=header, schema=schema, sep=sep
86             )
87         except AnalysisException as e:
```

2.**Orientação a objetos**

```

1  # src/processing/transformations.py
2  from pyspark.sql import DataFrame
3  from pyspark.sql import functions as F
4
5  class Transformation:
6      """
7      Nossa lógica
8      """
9
10     ## Adicionando no relatório o valor total do pedido
11
12     def avaliacao_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
13
14     def add_valor_total_pedidos(self, df_pedidos: DataFrame) -> DataFrame:
15
16     def filtra_pagamentos(self, df_pagamentos: DataFrame) -> DataFrame:
17
18     def filtra_pedidos(self, df_pedidos_filtrado: DataFrame) -> DataFrame:
19
20     def join_pagamentos_pedidos(
21         self, df_pagamento_filtrado: DataFrame, df_pedidos_filtrado: DataFrame
22     ) -> DataFrame:
23
24     def ordenar(self, df_relatorio: DataFrame) -> DataFrame:
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46

```

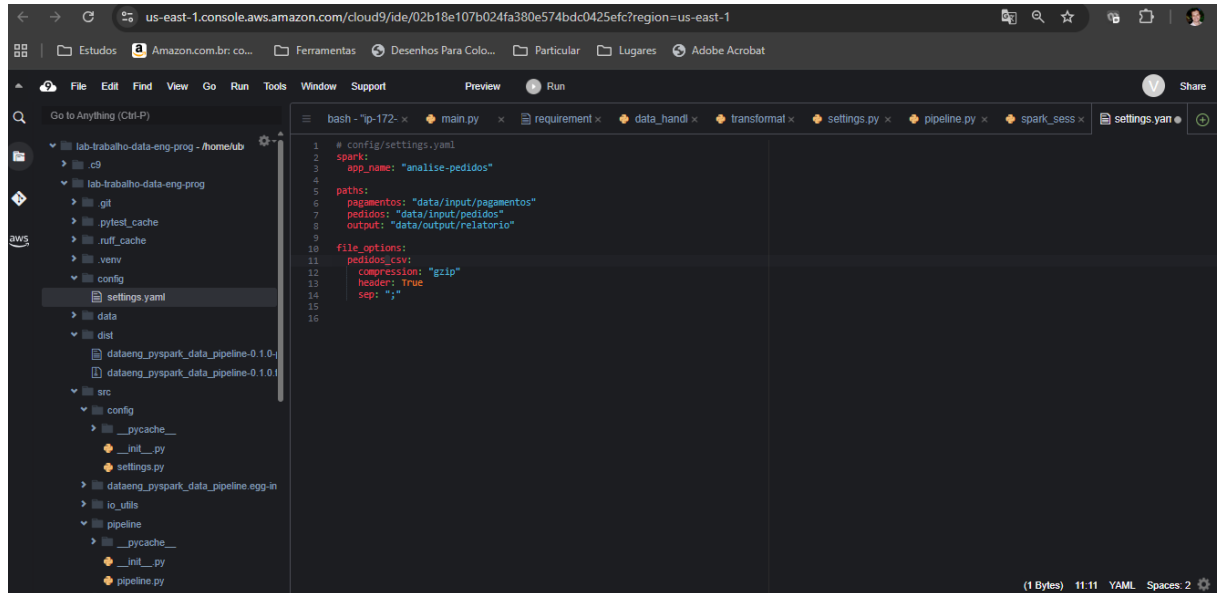
3.**Injeção de Dependências**

```

1  # src/main.py
2  from config.settings import carregar_config
3  from session.spark_session import SparkSessionManager
4  from pipeline.pipeline import Pipeline
5  import logging
6
7  logger = logging.getLogger(__name__)
8
9
10 # crie a configuração do logging
11 def configurar_logging():
12     """Configura o logging para todo o projeto."""
13     logging.basicConfig(
14         # Nível mínimo de severidade para ser registrado.
15         # DEBUG < INFO < WARNING < ERROR < CRITICAL
16         level=logging.INFO,
17         # Formato da mensagem de log.
18         format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
19         datefmt="%Y-%m-%d %H:%M:%S",
20         # Lista de handlers. Aqui, estamos logando para um arquivo e para o console.
21         handlers=[
22             logging.FileHandler("dataeng-pyspark-poo.log"), # log para arquivo
23             logging.StreamHandler(), # log para o console (terminal)
24         ],
25     )
26     logging.info("Logging configurado.")
27
28
29 def main():
30     """
31     Função principal que atua como a "Raiz de Composição".
32     Configura e executa o pipeline.
33     """
34
35     config = carregar_config()
36     app_name = config["spark"]["app_name"]
37
38     # 1. Inicialização da sessão Spark
39     spark = None
40     try:

```

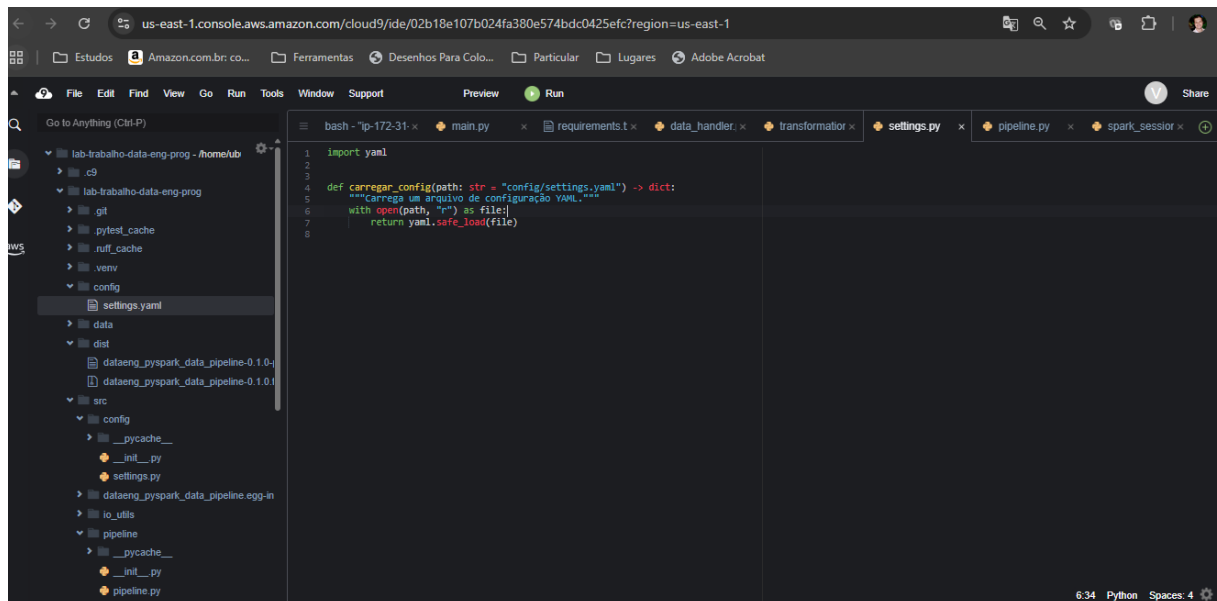
- Classes de configuração



```

1 # config/settings.yaml
2 spark:
3   app_name: "analise-pedidos"
4
5   paths:
6     pagamentos: "data/input/pagamentos"
7     pedidos: "data/input/pedidos"
8     output: "data/output/relatorio"
9
10  file_options:
11    pedidos_csv:
12      compression: "gzip"
13      header: true
14      sep: ";"
15
16

```



```

1 import yaml
2
3
4 def carregar_config(path: str = "config/settings.yaml") -> dict:
5     """Carrega um arquivo de configuração YAML."""
6     with open(path, "r") as file:
7         return yaml.safe_load(file)
8

```

- Classes de gerenciamento de sessão spark

```

1 # src/session/spark_session.py
2 from pyspark.sql import SparkSession
3
4 class SparkSessionManager:
5     """
6     Gerencia a criação e o acesso à sessão Spark.
7     """
8
9     @staticmethod
10     def get_spark_session(app_name: str = "analise-pedidos") -> SparkSession:
11         """
12         Cria e retorna uma sessão Spark.
13
14         :param app_name: Nome da aplicação Spark.
15         :return: Instância da SparkSession.
16         """
17         return SparkSession.builder.appName(app_name).master("local[*]").getOrCreate()
18
19

```

- Classes de leitura e escrita de dados

```

1 # src/io/utils/data_handler.py
2 import logging
3 from pyspark.errors import AnalysisException
4 from pyspark.sql import SparkSession, DataFrame
5 from pyspark.sql.types import (
6     StructType,
7     StructField,
8     StringType,
9     LongType,
10     ArrayType,
11     DateType,
12     FloatType,
13     DoubleType,
14     BooleanType,
15     TimestampType,
16 )
17
18 logger = logging.getLogger(__name__)
19
20
21 class DataHandler:
22     """
23     Classe responsável pela leitura (input) e escrita (output) de dados.
24     """
25
26     def __init__(self, spark: SparkSession):
27         pass
28
29     def _get_schema_pagamentos(self) -> StructType:
30         pass
31
32     def _get_schema_pedidos(self) -> StructType:
33         pass
34
35     def load_pagamentos(self, path: str) -> DataFrame:
36         pass
37
38     def load_pedidos(
39         self, path: str, compression: str, header: bool, sep: str
40     ) -> DataFrame:
41         pass
42
43     def write_parquet(self, df: DataFrame, path: str):
44         pass
45
46

```

- Classes de lógica de negócios

```

1 # src/processing/transformations.py
2 from pyspark.sql import DataFrame
3 from pyspark.sql import functions as F
4
5
6
7 class Transformation:
8     """
9     Wossa lógica
10    """
11
12    ## Adicionando no relatório o valor total do pedido
13
14    def avaliacao_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
15
16
17    def add_valor_total_pedidos(self, df_pedidos: DataFrame) -> DataFrame:
18
19
20    def filtra_pagamentos(self, df_pagamentos: DataFrame) -> DataFrame:
21
22
23    def filtra_pedidos(self, df_pedidos_filtrado: DataFrame) -> DataFrame:
24
25
26    def join_pagamentos_pedidos(
27        self, df_pagamento_filtrado: DataFrame, df_pedidos_filtrado: DataFrame
28    ) -> DataFrame:
29
30
31
32    def ordenar(self, df_relatorio: DataFrame) -> DataFrame:
33
34
35
36
37
38
39
40
41
42
43
44
45
46

```

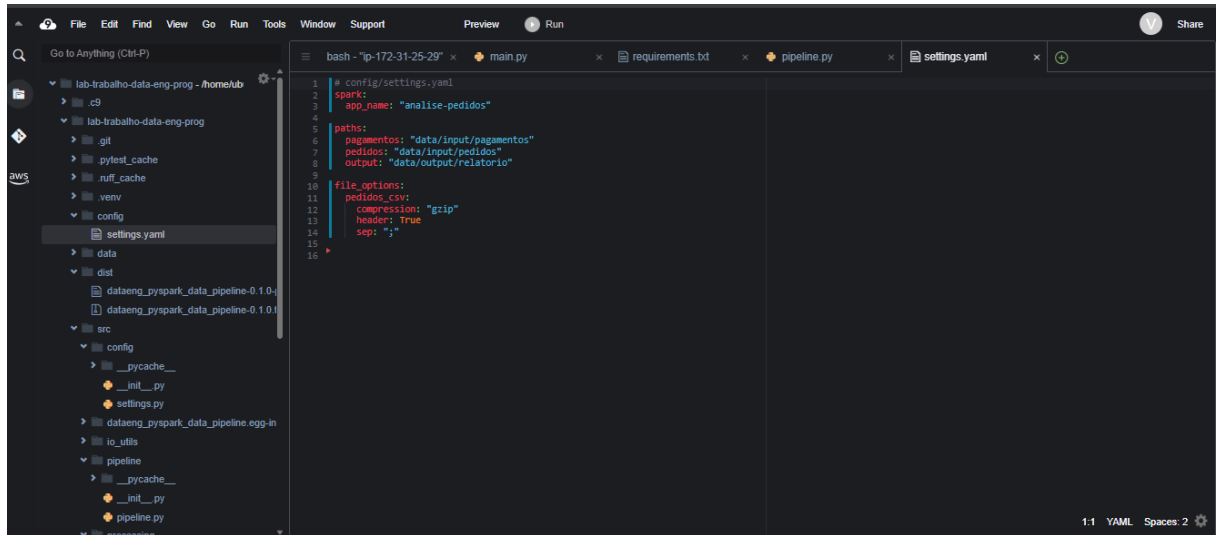
- **Classes de orquestração do pipeline**

```

1 # src/pipeline/pipeline.py
2 import logging
3 from pyspark.sql import SparkSession
4 from io_utils.data_handler import DataHandler
5 from processing.transformations import Transformation
6 import config.settings as settings
7
8 logger = logging.getLogger(__name__)
9
10
11 class Pipeline:
12     """
13     Encapsula a lógica de execução do pipeline de dados.
14    """
15
16    def __init__(self, spark: SparkSession):
17        self.spark = spark
18        self.data_handler = DataHandler(self.spark)
19        self.transformer = Transformation()
20
21
22    def run(self, config):
23        """
24        Executa o pipeline completo: carga, transformação, e salvamento.
25        """
26        logger.info("Pipeline iniciado...")
27
28        logger.info("Abrindo o dataframe de pagamentos")
29        path_pagamentos = config["paths"]["pagamentos"]
30        try:
31            df_pagamentos = self.data_handler.load_pagamentos(path=path_pagamentos)
32        except Exception as e:
33            logger.error(f"Problemas ao carregar dados de Pagamentos: {e}")
34            return # Interrompe o pipeline se os pedidos não puderem ser carregados
35
36        df_pagamentos.show(5, truncate=False)
37
38        logger.info("Abrindo o dataframe de pedidos")
39        path_pedidos = config["paths"]["pedidos"]
40        compression_pedidos = config["file_options"]["pedidos_csv"]["compression"]
41        header_pedidos = config["file_options"]["pedidos_csv"]["header"]
42
43
44
45
46

```

4.**Configurações centralizadas**

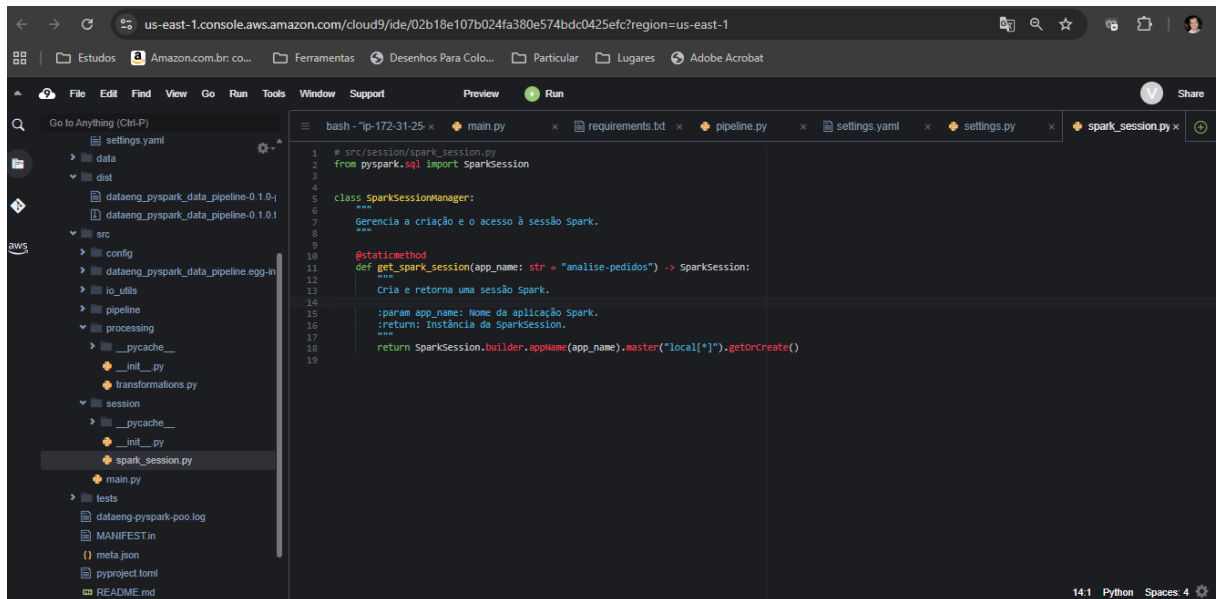


```

1 # config/settings.yaml
2 spark:
3   app_name: "analise-pedidos"
4
5 paths:
6   pagamentos: "data/input/pagamentos"
7   pedidos: "data/input/pedidos"
8   output: "data/output/relatorio"
9
10 file_options:
11   pedidos_csv:
12     compression: "gzip"
13     header: True
14     sep: ";"
15
16

```

5.**Sessão Spark**

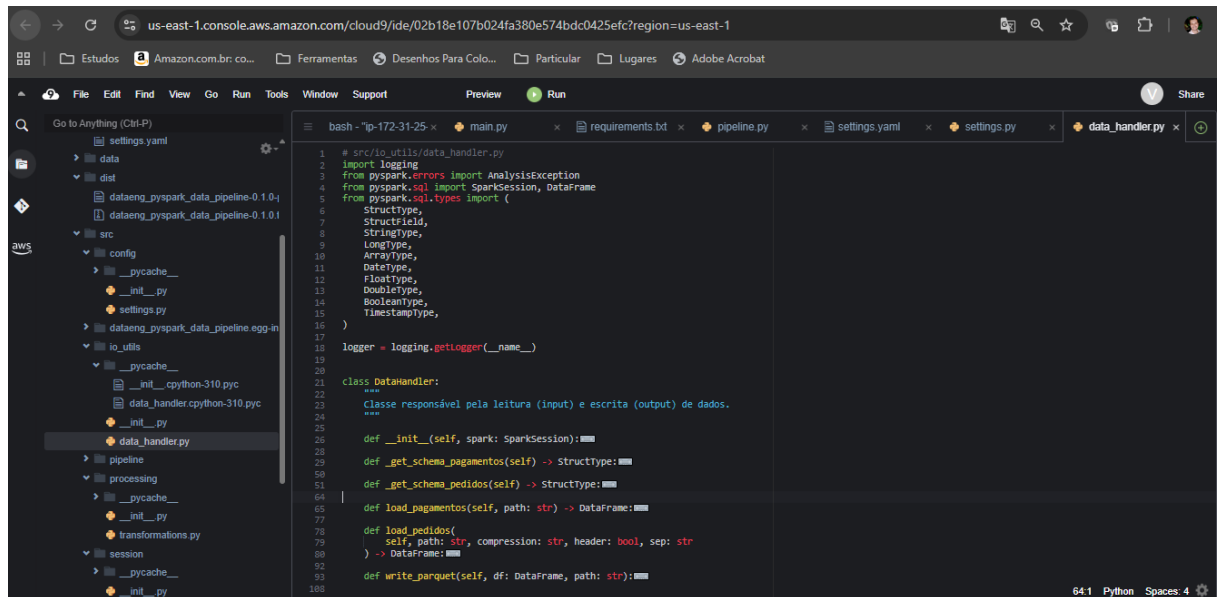


```

1 # src/session/spark_session.py
2 from pyspark.sql import SparkSession
3
4
5 class SparkSessionManager:
6   """
7   Gerencia a criação e o acesso à sessão Spark.
8   """
9
10   @staticmethod
11   def get_spark_session(app_name: str = "analise-pedidos") -> SparkSession:
12     """
13     Cria e retorna uma sessão Spark.
14
15     :param app_name: Nome da aplicação Spark.
16     :return: Instância da SparkSession.
17     """
18     return SparkSession.builder.appName(app_name).master("local[*]").getOrCreate()
19

```


6.**Leitura e Escrita de Dados (I/O)**

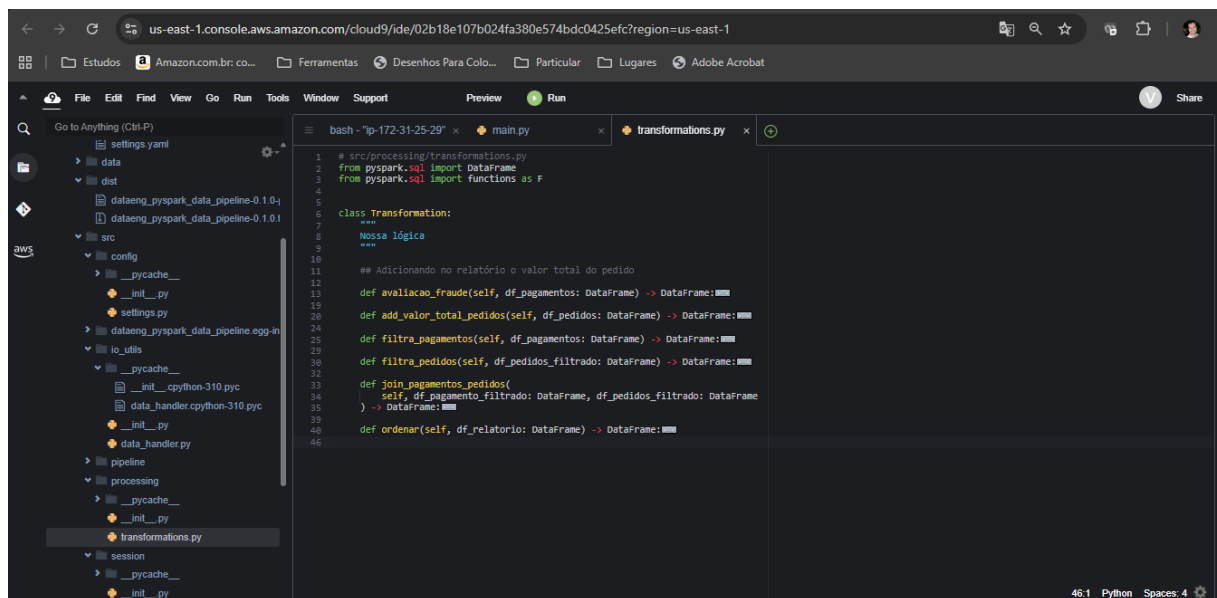


```

1 # src/io_utils/data_handler.py
2 import logging
3 from pyspark.errors import AnalysisException
4 from pyspark.sql import SparkSession, DataFrame
5 from pyspark.sql.types import (
6     StructType,
7     StructField,
8     StringType,
9     LongType,
10     ArrayType,
11     DateType,
12     FloatType,
13     DoubleType,
14     BooleanType,
15     TimestampType,
16 )
17
18 logger = logging.getLogger(__name__)
19
20
21 class DataHandler:
22     """
23     Classe responsável pela leitura (input) e escrita (output) de dados.
24     """
25
26     def __init__(self, spark: SparkSession):
27
28
29     def _get_schema_pagamentos(self) -> StructType:
30
31
32     def _get_schema_pedidos(self) -> StructType:
33
34
35     def load_pagamentos(self, path: str) -> DataFrame:
36
37
38     def load_pedidos(
39         self, path: str, compression: str, header: bool, sep: str
40     ) -> DataFrame:
41
42
43     def write_parquet(self, df: DataFrame, path: str):

```

7.**Lógica de Negócio**



```

1 # src/processing/transformations.py
2 from pyspark.sql import DataFrame
3 from pyspark.sql import functions as F
4
5
6 class Transformation:
7     """
8     Nossa lógica
9     """
10
11     ## Adicionando no relatório o valor total do pedido
12
13     def _avaliacao_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
14
15
16     def _add_valor_total_pedidos(self, df_pedidos: DataFrame) -> DataFrame:
17
18
19     def _filtra_pagamentos(self, df_pagamentos: DataFrame) -> DataFrame:
20
21
22     def _filtra_pedidos(self, df_pedidos_filtrado: DataFrame) -> DataFrame:
23
24
25     def _join_pagamentos_pedidos(
26         self, df_pagamento_filtrado: DataFrame, df_pedidos_filtrado: DataFrame
27     ) -> DataFrame:
28
29
30     def _ordenar(self, df_relatorio: DataFrame) -> DataFrame:

```

8.**Orquestração do pipeline**

```

6 import config.settings as settings
7
8 logger = logging.getLogger(__name__)
9
10
11 class Pipeline:
12     """
13     Encapsula a lógica de execução do pipeline de dados.
14     """
15
16     def __init__(self, spark: SparkSession):
17         self.spark = spark
18         self.data_handler = DataHandler(self.spark)
19         self.transformer = Transformation()
20
21     def run(self, config):
22         """
23         Executa o pipeline completo: carga, transformação, e salvamento.
24         """
25         logger.info("Pipeline iniciado...")
26
27         logger.info("Abrindo o dataframe de pagamentos")
28         path_pagamentos = config["paths"]["pagamentos"]
29         try:
30             df_pagamentos.show(5, truncate=False)
31         except Exception as e:
32             logger.error(f"Problemas ao carregar dados de pagamentos: {e}")
33             return # Interrompe o pipeline se os dados não puderem ser carregados
34
35         logger.info("Abrindo o dataframe de pedidos")
36         path_pedidos = config["paths"]["pedidos"]
37         compression_pedidos = config["file_options"]["pedidos_csv"]["compression"]
38         header_pedidos = config["file_options"]["pedidos_csv"]["header"]
39         separator_pedidos = config["file_options"]["pedidos_csv"]["sep"]
40         try:
41             # ... (rest of the code)
42         except Exception as e:
43             logger.error(f"Problemas ao carregar dados de pedidos: {e}")
44             return
45
46         logger.info("Avaliando Fraude")
47         try:

```

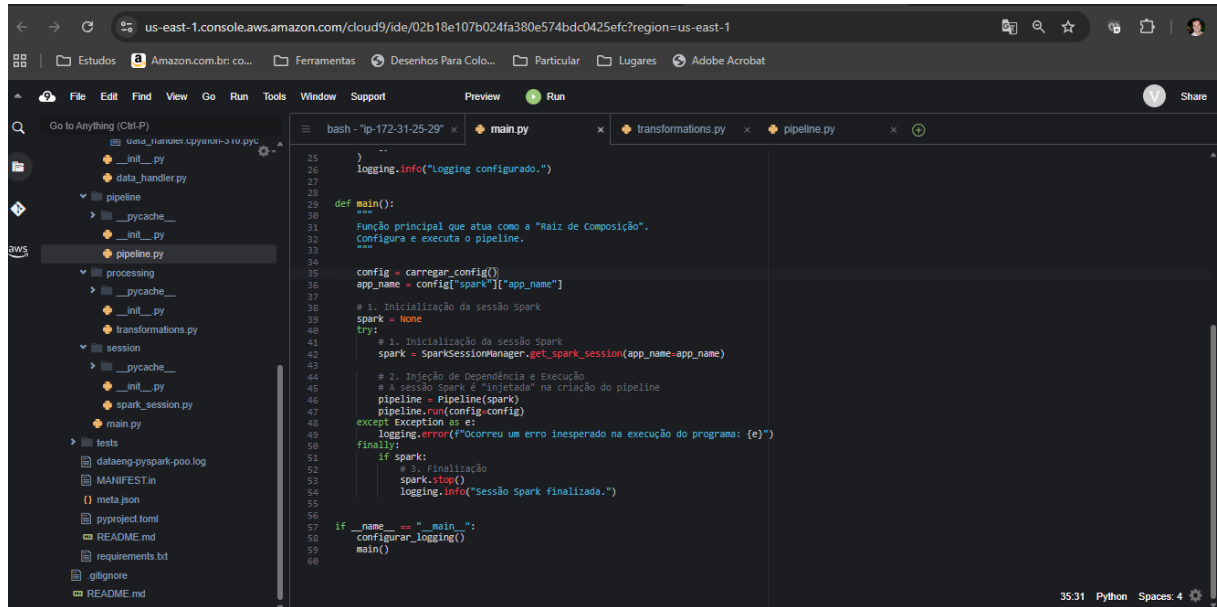
9. **Logging**

```

1 # src/main.py
2 from config.settings import carregar_config
3 from session.spark_session import SparkSessionManager
4 from pipeline.pipeline import Pipeline
5 import logging
6
7 logger = logging.getLogger(__name__)
8
9
10 # Crie a configuração do logging
11 def configurar_logging():
12     """Configura o logging para todo o projeto."""
13     logging.basicConfig(
14         # Nível mínimo de severidade para ser registrado.
15         # DEBUG < INFO < WARNING < ERROR < CRITICAL
16         level=logging.INFO,
17         # Formato da mensagem de log.
18         format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
19         datefmt='%Y-%m-%d %H:%M:%S',
20         # Lista de handlers. Aqui, estamos logando para um arquivo e para o console.
21         handlers=[
22             logging.FileHandler("dataeng-pyspark-poo.log"), # Log para arquivo
23             logging.StreamHandler(), # Log para o console (terminal)
24         ],
25     )
26     logging.info("Logging configurado.")
27
28
29 def main():
30     """
31     Função principal que atua como a "Raiz de Composição".
32     Configura e executa o pipeline.
33     """
34
35     config = carregar_config()
36     app_name = config["spark"]["app_name"]
37
38     # 1. Inicialização da sessão Spark
39     spark = None
40     try:

```

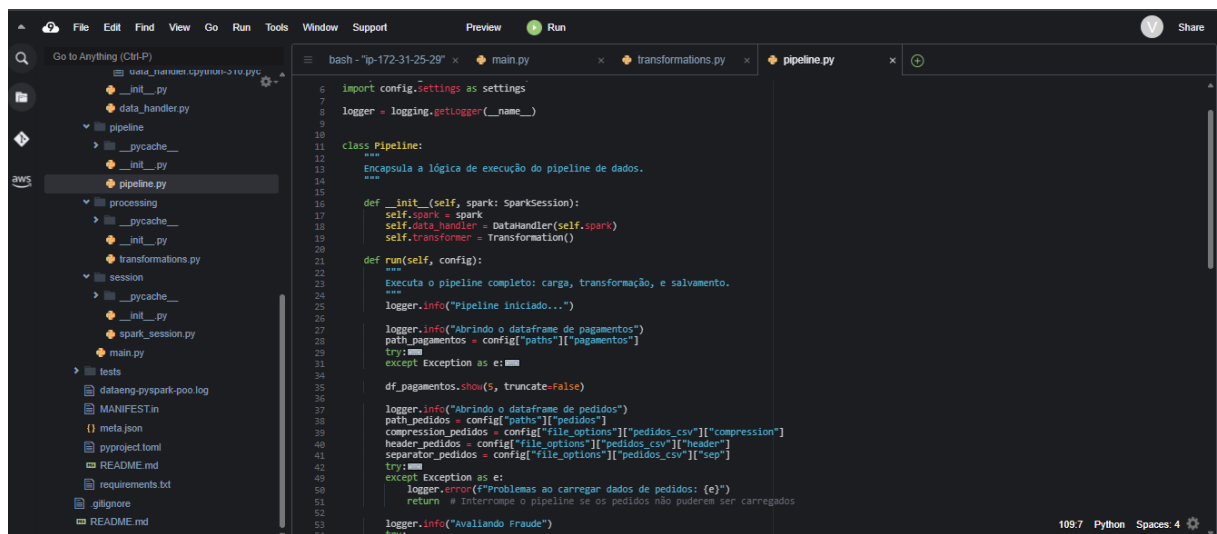
10.**Tratamento de Erros**



```

25 )
26 logging.info("Logging configurado.")
27
28 def main():
29     """
30     Função principal que atua como a "Rali de Composição".
31     Configura e executa o pipeline.
32     """
33
34     config = carregar_config()
35     app_name = config["spark"]["app_name"]
36
37     # 1. Inicialização da sessão spark
38     spark = None
39     try:
40         # 1. Inicialização da sessão spark
41         spark = SparkSessionManager.get_spark_session(app_name=app_name)
42
43         # 2. Injeção de dependência e execução
44         # A sessão Spark é "injetada" na criação do pipeline
45         pipeline = Pipeline(spark)
46         pipeline.run(config=config)
47     except Exception as e:
48         logging.error(f"Ocorreu um erro inesperado na execução do programa: {e}")
49     finally:
50         if spark:
51             # 3. Finalização
52             spark.stop()
53             logging.info("Sessão spark finalizada.")
54
55 if __name__ == "__main__":
56     configurar_logging()
57     main()
58

```



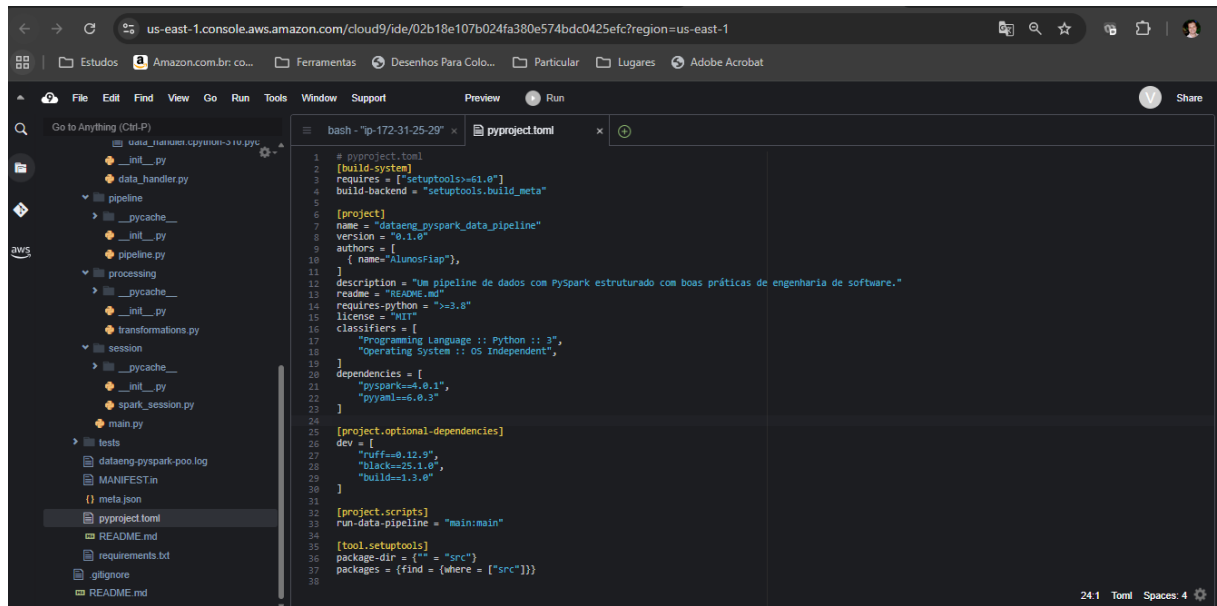
```

6 import config.settings as settings
7 logger = logging.getLogger(__name__)
8
9
10 class Pipeline:
11     """
12     Encapsula a lógica de execução do pipeline de dados.
13     """
14
15     def __init__(self, spark: SparkSession):
16         self.spark = spark
17         self.data_handler = DataHandler(self.spark)
18         self.transformer = Transformation()
19
20     def run(self, config):
21         """
22         Executa o pipeline completo: carga, transformação, e salvamento.
23         """
24         logger.info("Pipeline iniciado...")
25
26         logger.info("Abrindo o dataframe de pagamentos")
27         path_pagamentos = config["paths"]["pagamentos"]
28         try:
29             df_pagamentos = self.data_handler.load(path_pagamentos)
30             df_pagamentos.show(5, truncate=False)
31
32         except Exception as e:
33             logger.error(f"Problemas ao carregar dados de pagamentos: {e}")
34             return # Interrompe o pipeline se os dados não puderem ser carregados
35
36         logger.info("Abrindo o dataframe de pedidos")
37         path_pedidos = config["paths"]["pedidos"]
38         compression_pedidos = config["file_options"]["pedidos_csv"]["compression"]
39         header_pedidos = config["file_options"]["pedidos_csv"]["header"]
40         separator_pedidos = config["file_options"]["pedidos_csv"]["sep"]
41         try:
42             df_pedidos = self.transformer.transform(df_pagamentos, path_pedidos, compression_pedidos, header_pedidos, separator_pedidos)
43             df_pedidos.write.csv(path_pedidos, compression=compression_pedidos, header=header_pedidos, sep=separator_pedidos)
44
45         except Exception as e:
46             logger.error(f"Problemas ao salvar dados de pedidos: {e}")
47             return # Interrompe o pipeline se os dados não puderem ser salvos
48
49         logger.info("Avaliando Fraude")
50

```

11.**Empacotamento da aplicação**

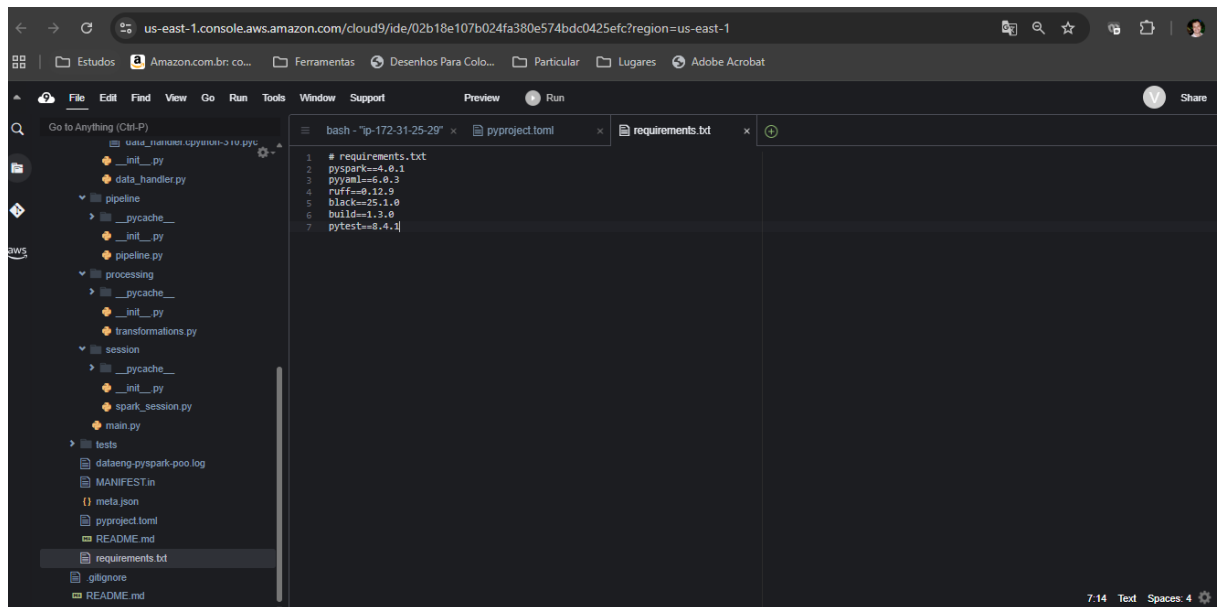
- CRIAR o arquivo `pyproject.toml`



The screenshot shows the AWS Cloud9 IDE interface. The left sidebar displays a file explorer with a project structure including folders like 'pipeline', 'processing', 'session', and 'tests', and files like 'dataeng_pyspark-poo.log', 'MANIFEST.in', 'meta.json', 'pyproject.toml', 'README.md', 'requirements.txt', and 'gitignore'. The main editor window is open to the 'pyproject.toml' file, which contains the following content:

```
1 # pyproject.toml
2 [build-system]
3 requires = ["setuptools>=61.0"]
4 build-backend = "setuptools.build_meta"
5
6 [project]
7 name = "dataeng_pyspark_data_pipeline"
8 version = "0.1.0"
9 authors = [
10     { name="AlunosFiap"},
11 ]
12 description = "Um pipeline de dados com Pyspark estruturado com boas práticas de engenharia de software."
13 readme = "README.md"
14 requires-python = ">=3.8"
15 license = "MIT"
16 classifiers = [
17     "Programming Language :: Python :: 3",
18     "Operating System :: OS Independent",
19 ]
20 dependencies = [
21     "pyspark==4.0.1",
22     "pyyaml==6.0.3"
23 ]
24
25 [project.optional-dependencies]
26 dev = [
27     "ruff==0.12.9",
28     "black==25.1.0",
29     "build==1.3.0"
30 ]
31
32 [project.scripts]
33 run-data-pipeline = "main:main"
34
35 [tool.setuptools]
36 package-dir = {"*" = "src"}
37 packages = {"find" = {"where" = ["src"]}}
```

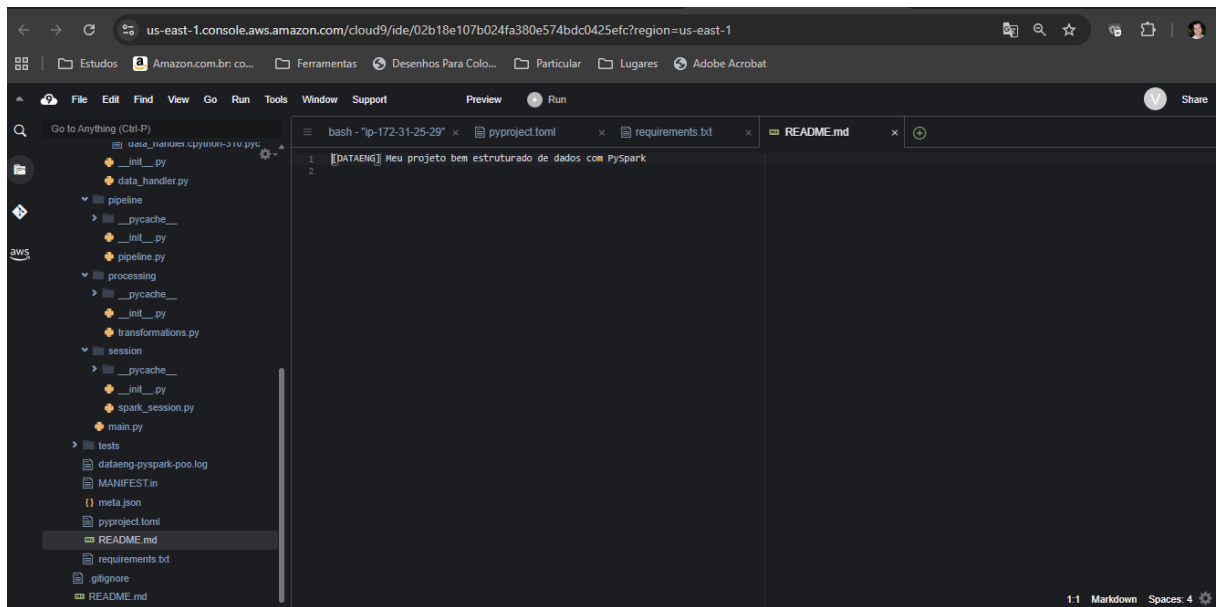
- CRIAR o arquivo `requirements.txt`



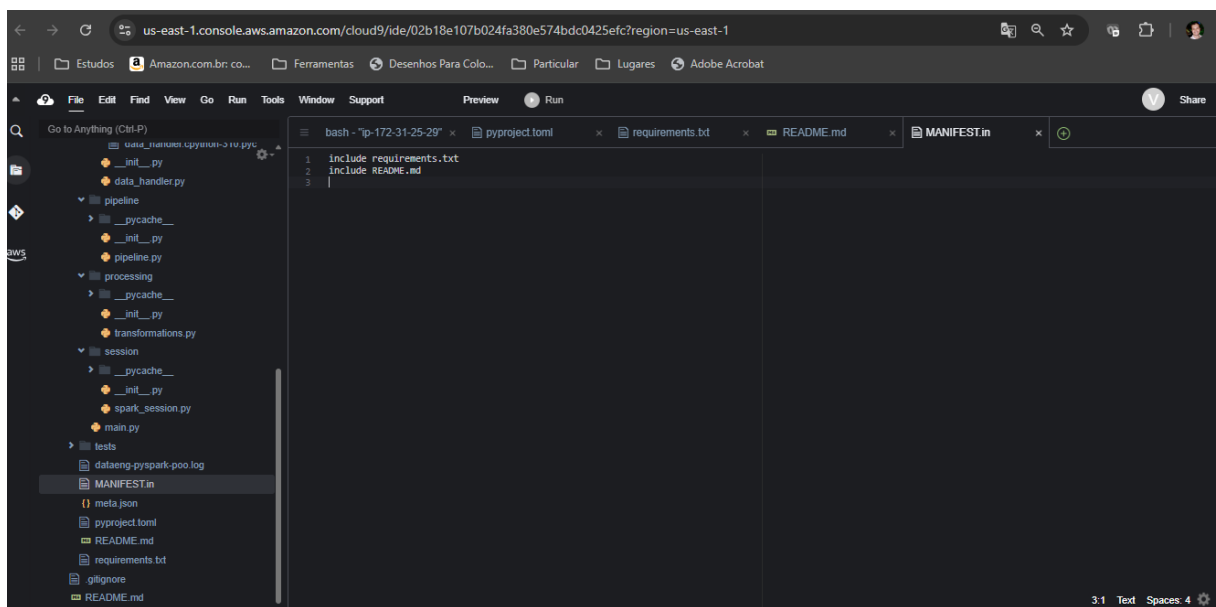
The screenshot shows the AWS Cloud9 IDE interface. The left sidebar displays the same file explorer as the previous screenshot, but now 'requirements.txt' is highlighted. The main editor window is open to the 'requirements.txt' file, which contains the following content:

```
1 # requirements.txt
2 pyspark==4.0.1
3 pyyaml==6.0.3
4 ruff==0.12.9
5 black==25.1.0
6 build==1.3.0
7 pytest==8.4.1
```

- CRIAR o arquivo `README.md`



- **CRIAR o arquivo `MANIFEST.in`**



12.**Testes unitários**

```
(.venv) voclabs:~/environment/lab-trabalho-data-eng-prog (main) $ pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.4.1, pluggy-1.6.0
rootdir: /home/ubuntu/environment/lab-trabalho-data-eng-prog
configfile: pyproject.toml
collected 1 item

tests/test_transformations.py . [100%]

===== 1 passed in 16.18s =====
(.venv) voclabs:~/environment/lab-trabalho-data-eng-prog (main) $
```



Relatório:

id_pedido	UF	forma_pagamento	VALOR_UNITARIO	DATA_CRIACAO
b8fd3cf8-4f14-4ebc-b1b6-72ff52bcd2e1	AL	PIX	1000.0	2025-05-28 12:23:52
6e9ff5a7-bccd-4545-94f8-0402ec204266	AL	PIX	1500.0	2025-05-27 13:39:12
d8b73bc3-a529-4741-960d-d5fd00f5771f	AL	PIX	700.0	2025-05-08 18:07:56
a66c83e0-16b1-49f9-b109-430fadfae281	AL	PIX	700.0	2025-04-17 21:07:23
64817981-2740-4b34-9ad3-2a5c719e20cb	AL	PIX	2500.0	2025-01-15 19:22:09
c093bbc9-bd56-43d0-afe2-5d8a52609bc9	AL	CARTAO_CREDITO	2500.0	2025-05-15 19:30:39
94a47e1e-e26f-4f80-9105-db822612d925	AL	CARTAO_CREDITO	1100.0	2025-05-15 17:31:56
d0493b71-6626-493a-8f3c-c1dae97e095e	AL	CARTAO_CREDITO	1000.0	2025-05-13 20:12:54
10a85986-e4da-446c-aed5-de1f5163a8ac	AL	CARTAO_CREDITO	900.0	2025-04-28 13:40:53
6076af29-68bc-4975-8ff3-89b0f5afa87a	AL	CARTAO_CREDITO	1100.0	2025-04-15 18:06:55
6a917788-20c6-4cfc-b51d-ececa0db13e1	AL	CARTAO_CREDITO	2500.0	2025-04-04 10:01:28
d8a94cc0-2df6-43ad-af0b-4ff5436a9581	AL	CARTAO_CREDITO	1000.0	2025-02-20 12:21:45
58ba0e74-c6d6-4beb-8a03-d32741db4628	BA	PIX	2000.0	2025-06-11 18:28:28
c7999417-4728-4ce4-bf2f-7e9f22ce46cb	BA	PIX	2500.0	2025-06-08 20:25:22
03cca111-f293-4221-bed5-83809ffa03b5	BA	PIX	1500.0	2025-05-26 13:17:38
e9e9689b-ceb0-4a03-b2d4-f7f70b861e23	BA	PIX	600.0	2025-05-19 13:45:57
ce122d01-c59c-4dad-9b9a-f1f84e3370a9	BA	PIX	700.0	2025-05-01 17:38:38
2372c444-6662-43d3-9ce4-bcbeb3564d2c	BA	PIX	900.0	2025-04-24 21:48:25
951e9872-2a32-47d7-bf35-6bab32840e3e	BA	PIX	1500.0	2025-03-26 20:58:11
cc8355ce-83f6-42c9-a779-418e55d9f70c	BA	PIX	900.0	2025-03-07 19:51:23

Log:

```
89 2025-11-26 21:31:32 - root - INFO - Logging configurado.
90 2025-11-26 21:31:38 - pipeline.pipeline - INFO - Pipeline iniciado...
91 2025-11-26 21:31:38 - pipeline.pipeline - INFO - Abrindo o dataframe de pagamentos
92 2025-11-26 21:31:42 - pipeline.pipeline - INFO - Abrindo o dataframe de pedidos
93 2025-11-26 21:31:43 - pipeline.pipeline - INFO - Avaliando Fraude
94 2025-11-26 21:31:43 - pipeline.pipeline - INFO - Adicionando no relatório o valor total do pedido
95 2025-11-26 21:31:44 - pipeline.pipeline - INFO - Filtrando Pagamentos
96 2025-11-26 21:31:45 - pipeline.pipeline - INFO - Filtrando Pedidos
97 2025-11-26 21:31:46 - pipeline.pipeline - INFO - Fazendo a junção dos dataframes
98 2025-11-26 21:31:48 - pipeline.pipeline - INFO - Ordena Resultado
99 2025-11-26 21:31:49 - pipeline.pipeline - INFO - Escrevendo o resultado em parquet
00 2025-11-26 21:31:52 - io_utils.data_handler - INFO - Dados salvos com sucesso em: data/output/relatorio
01 2025-11-26 21:31:52 - root - INFO - Sessão Spark finalizada.
02 2025-11-26 21:31:52 - py4j.clientserver - INFO - Closing down clientserver connection|
```

Desenvolvimento compacto

Github entrega: <https://github.com/BrunoGomesNogueira/aws-cloud9-trabalho/tree/project-review-herivelto>

1.**Schemas explícitos**

```

def _get_schema_pagamentos(self) -> StructType:
    """Define e retorna o schema para o dataframe dos Pagamentos."""
    return StructType(
        [
            StructField("id_pedido", StringType(), True),
            StructField("forma_pagamento", StringType(), True),
            StructField("valor_pagamento", DoubleType(), True),
            StructField("status", BooleanType(), True),
            StructField("data_processamento", TimestampType(), True),
            StructField(
                "avaliacao_fraude",
                StructType(
                    [
                        StructField("fraude", BooleanType(), True),
                        StructField("score", DoubleType(), True),
                    ]
                ),
                True,
            ),
        ]
    )

def _get_schema_pedidos(self) -> StructType:
    """Define e retorna o schema para o dataframe de pedidos."""
    return StructType(
        [
            StructField("ID_PEDIDO", StringType(), True),
            StructField("PRODUTO", StringType(), True),
            StructField("VALOR_UNITARIO", DoubleType(), True),
            StructField("QUANTIDADE", LongType(), True),
            StructField("DATA_CRIACAO", TimestampType(), True),
            StructField("UF", StringType(), True),
            StructField("ID_CLIENTE", LongType(), True),
        ]
    )

```

2.**Orientação a objetos**

```

15 class Transformation:
16 > def avaliacao_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
17     )
18
19     # ===== Colunas Calculadas =====
20
21 > def add_valor_total_pedidos(self, df_pedidos: DataFrame) -> DataFrame:
22     return df_pedidos.withColumn("VALOR_TOTAL", F.col("VALOR_UNITARIO") * F.col("QUANTIDADE"))
23
24     # ===== Filtros =====
25
26 > def filtra_pagamentos_aprovados_sem_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
27     return df_pagamentos.filter(condicao)
28
29 > def filtra_pedidos_2025(self, df_pedidos: DataFrame) -> DataFrame:
30     return df_pedidos.filter(condicao)
31
32     # ===== Condições Reutilizáveis =====
33
34     @staticmethod
35 > def _criar_condicao_pagamento_valido_sem_fraude() -> Column:
36     return (F.col("status") == False) & (F.col("fraude") == False)
37
38     @staticmethod
39 > def _criar_condicao_ano(ano: int) -> Column:
40     return F.year(F.col("DATA_CRIACAO")) == ano
41
42     # ===== Joins =====
43
44 > def join_pagamentos_pedidos(self, df_pagamentos: DataFrame, df_pedidos: DataFrame) -> DataFrame:
45     return df_pagamentos.join(df_pedidos, on="ID_PEDIDO", how="inner").select("colunas_selecionadas")
46
47     # ===== Ordenação =====
48
49 > def ordenar_por_valor_pagamento_data_criacao(self, df_relatorio: DataFrame) -> DataFrame:
50     )
51

```

3.**Injeção de Dependências**

```

1 # src/main.py
2 from core.config import load_settings
3 from core.spark_session import SparkSessionManager
4 from core.logger import setup_logging, get_logger
5 from pipeline import Pipeline
6
7 # Obter logger para este módulo
8 logger = get_logger(__name__)
9
10
11 def main():
12     """
13     Função principal que atua como a "Raiz de Composição".
14     Configura e executa o pipeline.
15     """
16
17     config = load_settings()
18     app_name = config.spark_app_name
19
20     # 1. Inicialização da sessão Spark
21     spark = None
22     try:
23         spark = SparkSessionManager.get_spark_session(app_name=app_name)
24
25         # 2. Injeção de Dependência e Execução
26         # A sessão Spark é "injetada" na criação do pipeline
27         pipeline = Pipeline(spark)
28         pipeline.run(config=config)
29     except Exception as e:
30         logger.error(f"Ocorreu um erro inesperado na execução do programa: {e}")
31     finally:
32         if spark:
33             # 3. Finalização
34             spark.stop()
35             logger.info("Sessão Spark finalizada.")
36

```


- **Classes de configuração**

```
Code Blame 74 lines (50 loc) · 1.69 KB
1 import yaml
2 from pydantic import BaseModel
3
4
5 class SparkConfig(BaseModel):
6     """Configurações do Spark."""
7     app_name: str
8
9
10
11 > class PathConfig(BaseModel):
12     output: str
13
14
15 > class PedidosCsvConfig(BaseModel):
16     sep: str
17
18
19
20 class FileOptionsConfig(BaseModel):
21     """Opções de arquivo."""
22     pedidos_csv: PedidosCsvConfig
23
24
25 > class Settings(BaseModel):
26     return cls(**config_data)
27
28
29
30 > def load_settings(path: str = "config/settings.yaml") -> Settings:
31     return Settings.from_yaml(path)
```

- **Classes de gerenciamento de sessão spark**

```
heri-macedo refactor folder structure
Code Blame 18 lines (14 loc) · 515 Bytes
1 # src/session/spark_session.py
2 from pyspark.sql import SparkSession
3
4
5 class SparkSessionManager:
6     """
7     Gerencia a criação e o acesso à sessão Spark.
8     """
9
10     @staticmethod
11     def get_spark_session(app_name: str = "analise-pedidos") -> SparkSession:
12         """
13         Cria e retorna uma sessão Spark.
14
15         :param app_name: Nome da aplicação Spark.
16         :return: Instância de SparkSession.
17         """
18         return SparkSession.builder.appName(app_name).master("local[*]").getOrCreate()
```

- **Classes de lógica de negócios**

```
15 class Transformation:
16     """
17     Classe responsável por todas as transformações de dados do pipeline.
18
19     Utiliza a Column API do PySpark para operações type-safe e otimizadas.
20     """
21
22     # ===== Colunas de Fraude =====
23
24 > def avaliar_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
25     )
26
27     # ===== Colunas Calculadas =====
28
29 > def add_valor_total_pedidos(self, df_pedidos: DataFrame) -> DataFrame:
30     return df_pedidos.withColumn("VALOR_TOTAL", F.col("VALOR_UNITARIO") * F.col("QUANTIDADE"))
31
32     # ===== Filtros =====
33
34 > def filtra_pagamentos_aprovados_sem_fraude(self, df_pagamentos: DataFrame) -> DataFrame:
35     return df_pagamentos.filter(condicao)
36
37 > def filtra_pedidos_2025(self, df_pedidos: DataFrame) -> DataFrame:
38     return df_pedidos.filter(condicao)
39
40     # ===== Condições Reutilizáveis =====
41
42     @staticmethod
43     def _criar_condicao_pagamento_valido_sem_fraude() -> Column:
44         return (F.col("status") == "PAGO") & (F.col("fraude") == False)
45
46     @staticmethod
47     def _criar_condicao_ano(ano: int) -> Column:
48         return F.year(F.col("DATA_CRIACAO")) == ano
49
50     # ===== Joins =====
```

- **Classes de orquestração do pipeline**



```
Code Blame 116 lines (94 loc) · 4.69 KB
1 # src/pipeline.py
2 from pyspark.sql import SparkSession
3 from core.logger import get_logger
4 from core.config import settings
5 from data.handlers import DataHandler
6 from data.transformations import Transformation
7
8 logger = get_logger(__name__)
9
10
11 class Pipeline:
12     """
13     Encapsula a lógica de execução do pipeline de dados.
14     """
15
16     def __init__(self, spark: SparkSession):
17         self.spark = spark
18         self.data_handler = DataHandler(self.spark)
19         self.transformation = Transformation()
20
21     > def run(self, config: Settings) -> None:
116         return # interrompe o pipeline se os dados não puderem ser carregados
```

4. **Configurações centralizadas**

Code Blame 15 lines (12 loc) · 257 Bytes

```
1 # config/settings.yaml
2 spark:
3   app_name: "analise-pedidos"
4
5 paths:
6   pagamentos: "data/input/pagamentos"
7   pedidos: "data/input/pedidos"
8   output: "data/output/relatorio"
9
10 file_options:
11   pedidos_csv:
12     compression: "gzip"
13     header: True
14     sep: ";"
```

5. **Sessão Spark**

Code Blame 18 lines (14 loc) · 515 Bytes

```
1 # src/session/spark_session.py
2 from pyspark.sql import SparkSession
3
4
5 class SparkSessionManager:
6     """
7     Gerencia a criação e o acesso à sessão Spark.
8     """
9
10     @staticmethod
11     def get_spark_session(app_name: str = "analise-pedidos") -> SparkSession:
12         """
13         Cria e retorna uma sessão Spark.
14
15         :param app_name: Nome da aplicação Spark.
16         :return: Instância da SparkSession.
17         """
18         return SparkSession.builder.appName(app_name).master("local[*]").getOrCreate()
```

6.**Leitura e Escrita de Dados (I/O)**

```

19
20
21 class DataHandler:
22     """
23     Classe responsável pela leitura (input) e escrita (output) de dados.
24     """
25
26     def __init__(self, spark: SparkSession):
27         self.spark = spark
28
29 > def _get_schema_pagamentos(self) -> StructType: ...
49
50
51 > def _get_schema_pedidos(self) -> StructType: ...
63
64
65 > def load_pagamentos(self, path: str) -> DataFrame: ...
77
78
79 > def load_pedidos(self, path: str, compression: str, header: bool, sep: str) -> DataFrame: ...
91
92
93 > def write_parquet(self, df: DataFrame, path: str): ...
109
raise

```

7.**Lógica de Negócio**

```

19 class Transformation:
20     """
21     Classe responsável por todas as transformações de dados do pipeline.
22     Utiliza a Column API do PySpark para operações type-safe e otimizadas.
23     """
24
25     # ===== Colunas de fraude =====
26
27 > def avaliar_fraude(self, df_pagamentos: DataFrame) -> DataFrame: ...
41
42
43     # ===== Colunas Calculadas =====
44
45 > def add_valor_total_pedidos(self, df_pedidos: DataFrame) -> DataFrame: ...
46
47     return df_pedidos.withColumn("VALOR_TOTAL", F.col("VALOR_UNITARIO") * F.col("QUANTIDADE"))
48
49     # ===== Filtragem =====
50
51 > def filtra_pagamentos_aprovados_sem_fraude(self, df_pagamentos: DataFrame) -> DataFrame: ...
52
53     return df_pagamentos.filter(condicao)
54
55 > def filtra_pedidos_2025(self, df_pedidos: DataFrame) -> DataFrame: ...
56
57     return df_pedidos.filter(condicao)
58
59     # ===== Condições Reutilizáveis =====
60
61 @staticmethod
62 def _criar_condicao_pagamento_valido_sem_fraude() -> Column: ...
63
64     return (F.col("status") == "false") & (F.col("fraude") == "false")
65
66 @staticmethod
67 def _criar_condicao_ano(ano: int) -> Column: ...
68
69     return F.year(F.col("DATA_CRIACAO")) == ano
70
71     # ===== Joins =====
72

```

8.**Orquestração do pipeline**

```
Code Blame 110 lines (94 loc) · 4.69 KB

1 # src/pipeline.py
2 from pyspark.sql import SparkSession
3 from core.logger import get_logger
4 from core.config import settings
5 from data.handlers import DataHandler
6 from data.transformations import Transformation
7
8 logger = get_logger(__name__)
9
10
11 class Pipeline:
12     """
13     Encapsula a lógica de execução do pipeline de dados.
14     """
15
16     def __init__(self, spark: SparkSession):
17         self.spark = spark
18         self.data_handler = DataHandler(self.spark)
19         self.transformer = Transformation()
20
21     def run(self, config: Settings) -> None:
22         """
23         return: interrompe o pipeline se os dados não puderem ser carregados
24         """
25         return
```

9.**Logging**

```
Code Blame 40 lines (32 loc) · 1.06 KB

1 # src/main.py
2 from core.config import load_settings
3 from core.spark_session import SparkSessionManager
4 from core.logger import setup_logging, get_logger
5 from pipeline import Pipeline
6
7 # Obter logger para este módulo
8 logger = get_logger(__name__)
9
10
11 def main():
12     """
13     Função principal que atua como a "Raiz de Composição".
14     Configura e executa o pipeline.
15     """
16
17     config = load_settings()
18     app_name = config.spark.app_name
19
20     # 1. Inicialização da sessão Spark
21     spark = None
22     try:
23         spark = SparkSessionManager.get_spark_session(app_name=app_name)
24
25     # 2. Injeção de Dependência e Execução
26     # A sessão Spark é "injetada" na criação do pipeline
27     pipeline = Pipeline(spark)
28     pipeline.run(config=config)
29     except Exception as e:
30         logger.error(f"Ocorreu um erro inesperado na execução do programa: {e}")
31     finally:
32         if spark:
33             # 3. Finalização
34             spark.stop()
35             logger.info("Sessão Spark finalizada.")
36
37
38 if __name__ == "__main__":
39     setup_logging()
40     main()
```

10.**Tratamento de Erros**

```
50         sep=separator_pedidos,
51     )
52     except Exception as e:
53         logger.error(f"Problemas ao carregar dados de pedidos: {e}")
54         return # Interrompe o pipeline se os pedidos não puderem ser carregados
55
56     logger.info("Avaliando Fraude")
57     try:
58         df_pagamentos_fraude = self.transformer.avaliacao_fraude(df_pagamentos)
59     except Exception as e:
60         logger.error(f"Problemas ao carregar dados de análise de fraude: {e}")
61         return # Interrompe o pipeline se os pedidos não puderem ser carregados
62
63     df_pagamentos_fraude.show(5, truncate=False)
64
65     logger.info("Adicionando no relatório o valor total do pedido")
66     try:
67         df_valor_total = self.transformer.add_valor_total_pedidos(df_pedidos)
68     except Exception as e:
69         logger.error(f"Problemas ao carregar dados com total dos pedidos: {e}")
70         return # Interrompe o pipeline se os pedidos não puderem ser carregados
71
72     df_valor_total.show(5, truncate=False)
73
74     logger.info("Filtrando Pagamentos")
75     try:
76         df_filtro_pagamento = self.transformer.filtro_pagamentos_aprovados_sem_fraude(df_pagamentos_fraude)
77     except Exception as e:
78         logger.error(f"Problemas ao carregar dados de pagamentos filtrados: {e}")
79         return # Interrompe o pipeline se os pedidos não puderem ser carregados
```

11.**Empacotamento da aplicação**

- CRIAR o arquivo `pyproject.toml`

```

1  # pyproject.toml
2  [build-system]
3  requires = ["setuptools>=61.0"]
4  build-backend = "setuptools.build_meta"
5
6  [project]
7  name = "dataeng_pyspark_data_pipeline"
8  version = "0.1.0"
9  authors = [
10   { name="AlunosFiap"},
11  ]
12  description = "Um pipeline de dados com PySpark estruturado com boas práticas de engenharia de software."
13  readme = "README.md"
14  requires-python = ">=3.8"
15  license = "MIT"
16  classifiers = [
17   "Programming Language :: Python :: 3",
18   "Operating System :: OS Independent",
19  ]
20  dependencies = [
21   "pyspark==4.0.1",
22   "pyyaml==6.0.3"
23  ]
24
25  [project.optional-dependencies]
26  dev = [
27   "ruff==0.12.9",
28   "black==25.1.0",
29   "build==1.3.0"
30  ]
31
32  [project.scripts]
33  run-data-pipeline = "main:main"
34
35  [tool.setuptools]
36  package-dir = {"" = "src"}
37  packages = {find = {where = ["src"]}}
38
39  [tool.black]
40  line-length = 120
41  target-version = ['py311']
42  skip-string-normalization = false
43  # Permite que Black preserve method chaining do PySpark
44  preview = true
45
46  [tool.ruff]
47  line-length = 120
48  target-version = "py311"
49
50  [tool.ruff.lint]
51  select = [
52   "E", # pycodestyle errors
53   "W", # pycodestyle warnings
54   "F", # pyflakes
55   "I", # isort
56   "N", # pep8-naming
57  ]
58  ignore = [

```

- CRIAR o arquivo `requirements.txt`

```
1 # requirements.txt
2 pyspark==4.0.1
3 pyyaml==6.0.3
4 pydantic==2.10.4
5 pydantic-settings==2.7.1
6 ruff==0.12.9
7 black==25.1.0
8 build==1.3.0
9 pytest==8.4.1
```

- CRIAR o arquivo `README.md`

```
1  # 📁 Projeto de Processamento de Dados com PySpark
2
3  Este repositório contém um pipeline de processamento de dados
4  desenvolvido em Python + PySpark, estruturado de forma modular
5  utilizando Programação Orientada a Objetos (POO). O projeto foi
6  projetado para ser executado em ambientes locais, como AWS Cloud9, e
7  utiliza dados de exemplo (JSON compactado) para demonstrar a leitura,
8  transformação e gravação em formatos otimizados.
9
10 -----
11
12  ## 🚀 Funcionalidades Principais
13
14  - 📄 Leitura de dados JSON compactados (.json.gz)
15  - ⚙️ Transformações organizadas em classes
16  - 🧪 Testes automatizados com `pytest`
17  - ⚙️ Configuração centralizada via `settings.yaml`
18  - 🏗️ Arquitetura limpa separando sessão Spark, I/O e transformações
19  - 💾 Gravação dos dados transformados em formatos otimizados (ex.:
20    Parquet)
21
22
23  ## 🛠️ Pré-requisitos
24
25  - Python 3.10+
26  - Java 8/11 (necessário para o Spark)
27
28  Instalação dos pacotes após criar o seu virtual environemnt:
29
30  ``` bash
31  make install
32  ```
33
34  -----
35
36  ## 📄 Como Executar
37
38  ### 1. Ativar ambiente virtual (opcional)
```


- CRIAR o arquivo `MANIFEST.in`

```
Code Blame 2 lines (2 loc) · 43 Bytes

1 include requirements.txt
2 include README.md
```

12.**Testes unitários**

```
54 class TestAddValorTotal:
55     """Testes para a função add_valor_total_pedidos."""
56
57     def test_calculo_valor_total_correto(self, transformer, df_pedidos_validos):
58         """
59         Testa se o valor total é calculado corretamente (preço x quantidade).
60         """
61         # Act
62         resultado = transformer.add_valor_total_pedidos(df_pedidos_validos)
63
64         # Assert
65         assert "VALOR_TOTAL" in resultado.columns
66
67         # Verificar cálculos específicos
68         dados = resultado.collect()
69         assert dados[0]["VALOR_TOTAL"] == 3500.00 # 3500 * 1
70         assert dados[1]["VALOR_TOTAL"] == 241.00 # 120.50 * 2
71         assert dados[2]["VALOR_TOTAL"] == 250.00 # 250 * 1
72
73     def test_valor_total_com_zero(self, transformer, spark_session, schema_pedidos):
74         """Testa cálculo quando preço ou quantidade é zero."""
75         # Arrange
76         dados = [
77             ("P001", "Produto", 0.0, 5, datetime(2025, 1, 1, 0, 0, 0), "SP", 101),
78             ("P002", "Produto", 100.0, 0, datetime(2025, 1, 1, 0, 0, 0), "RJ", 102),
79         ]
80         df = spark_session.createDataFrame(dados, schema_pedidos)
81
82         # Act
83         resultado = transformer.add_valor_total_pedidos(df)
84
85         # Assert
86         dados_resultado = resultado.collect()
87         assert dados_resultado[0]["VALOR_TOTAL"] == 0.0 # 0 * 5
88         assert dados_resultado[1]["VALOR_TOTAL"] == 0.0 # 100 * 0
89
90     def test_valor_total_com_valores_grandes(self, transformer, df_pedidos_edge_cases):
91         """Testa cálculo com valores muito grandes."""
92         # Act
93         resultado = transformer.add_valor_total_pedidos(df_pedidos_edge_cases)
94
95         # Assert
96         registro_grande = resultado.filter(F.col("ID_PEDIDO") == "P008").first()
97         assert registro_grande["VALOR_TOTAL"] == 99999990.0 # 99999.99 * 1000
```

FIAP